ลำดับขั้นตอนทั้งหมด — ระบบจองคิวและนัดหมายออนไลน์

- 👤 **เรา** = คนที่ 1 (เจ้าของ repo / เครื่องหลัก) · 👥 **เพื่อน** = คนที่ 2–4
- ทุกคำสั่งใช้ใน **PowerShell** ได้ และต้องรันใน **โฟลเดอร์โปรเจกต์** (โฟลเดอร์ที่มี `docker-compose.yml`)

ช่วงที่ 1 — ส่งงานให้เพื่อน

👤 เรา

1. ส่ง `queue-booking.zip` พร้อมข้อความแจ้งในกลุ่ม (อยากให้เปิดอ่านทันที ส่ง `docs\DEMO-SCRIPT.pdf` และ `docs\COMMANDS.pdf` แยกด้วย)
2. ส่ง **รหัสผ่าน Radmin** ของ network `queue-booking-team-4821` ให้เพื่อนแต่ละคนทางแชตส่วนตัว

👥 เพื่อน

1. ติดตั้ง Docker Desktop, Postman และ Radmin VPN (โหลดจาก radmin-vpn.com เท่านั้น)
2. แตก zip แล้วรันในโฟลเดอร์ `queue-booking` (บรรทัดแรกทำครั้งแรกครั้งเดียว — สร้างไฟล์ `.env` ที่เก็บรหัสผ่าน):

```
copy .env.example .env
docker compose up -d --build
```

3. เช็กว่าระบบขึ้น (ต้องได้ `{"status":"ok"}`):

```
curl.exe http://localhost:8081/health
```

4. Import `postman/queue-booking.postman_collection.json` เข้า Postman → กด **Run collection** ทุก request ต้องผ่าน
5. อ่าน `README.md`, `docs/DEMO-SCRIPT.pdf` และโค้ดในโฟลเดอร์ของตัวเอง

ช่วงที่ 2 — ต่อ Radmin (เมื่อจะเทสต์ร่วมกัน)

👥 เพื่อน

1. เปิด Radmin → **Network** → **Join an existing network** → กรอก `queue-booking-team-4821` + รหัสผ่าน
2. ส่ง IP `26.x.x.x` ของตัวเองมาให้เรา

👤 เรา

1. เช็กในโปรแกรม Radmin ว่าเห็นชื่อเพื่อนและจุดหน้าชื่อเป็นสีเขียว
2. เปิด PowerShell แบบ **Run as administrator**: กด Start → พิมพ์ `PowerShell` → คลิกขวา → Run as administrator → Yes (หัวหน้าต่างต้องขึ้น "Administrator")
3. เปิด firewall (แทน `26.x.x.x,...` ด้วย IP ของเพื่อน คั่นด้วย comma ห้ามมีช่องว่าง):

```
New-NetFirewallRule -DisplayName "Queue Booking API (Radmin)" -Direction Inbound -Protocol TCP -LocalPort 8081-8084 -Action Allow -Profile Any -InterfaceAlias "Radmin VPN" -RemoteAddress 26.x.x.x,26.y.y.y,26.z.z.z
```

4. เช็กว่าสร้างสำเร็จ (ต้องเห็น `True` และ `Allow`):

```
Get-NetFirewallRule -DisplayName "Queue Booking API (Radmin)" | Select-Object DisplayName, Enabled, Action
```

5. เปิดระบบ แล้วบอกในกลุ่มว่า "เปิดแล้ว":

```
docker compose up -d
```

👥 **เพื่อน**

1. ทดสอบ (ต้องได้ `{"status":"ok"}`):

```
curl.exe http://26.151.109.184:8081/health
```

2. Postman → แท็บ **Variables** ของ collection → เปลี่ยน `localhost` เป็น `26.151.109.184` ทั้ง 4 ตัว → Run collection ใหม่ (เลิกใช้แล้วเปลี่ยนกลับเป็น `localhost`)

👤 **เรา — ใช้เสร็จแล้วต้องปิดทุกครั้ง** (PowerShell แบบ Administrator):

```
Remove-NetFirewallRule -DisplayName "Queue Booking API (Radmin)"
```

ช่วงที่ 3 — เพื่อนแก้งานแล้วส่งกลับ (วนซ้ำจนเสร็จ)

👥 **เพื่อน**

1. แก้เฉพาะโฟลเดอร์ของตัวเอง (จะแก้ไฟล์ส่วนกลาง หรือเปลี่ยนรูปแบบ JSON ของ API ต้องบอกในกลุ่มก่อน)
2. ก่อนส่ง เช็กว่า `docker compose up -d --build` ผ่าน และ Postman รันผ่านหมด
3. zip ส่งกลับเฉพาะโฟลเดอร์ของตัวเอง เช่น `booking-service.zip` พร้อมบอกสั้นๆ ว่าแก้อะไร

👤 **เรา (ทำทีละคน)**

1. **ลบ** โฟลเดอร์ของ service นั้นทิ้งก่อน แล้ววางโฟลเดอร์ที่เพื่อนส่งมาแทน (วางทับเฉยๆ ไฟล์ที่เพื่อนลบไปจะยังค้าง)
2. build และรันใหม่:

```
docker compose up -d --build
```

3. รันเทสต์ (ต้องไม่มี FAIL):

```
& "C:\Program Files\Git\bin\bash.exe" -c './scripts/test-auth.sh &
& ./scripts/test-service.sh && ./scripts/test-booking.sh && ./scri
pts/test-notification.sh'
```

ถ้าเพื่อนตั้งใจเปลี่ยนพฤติกรรมของ API เทสต์อาจ fail ตามที่คาดไว้ — ถามเพื่อนก่อนว่าตั้งใจหรือเปล่า

4. commit แล้วสร้าง zip ใหม่:

```
git add -A; git commit -m "Update booking-service from member 3";
if ($?) { git archive --format=zip --prefix=queue-booking/ -o ../q
ueue-booking.zip HEAD }
```

5. ส่ง zip ใหม่ให้ทุกคน เพื่อให้ใช้โค้ดเวอร์ชันเดียวกัน

ช่วงที่ 4 — ซ้อม Demo

👥 ทุกคน

1. อ่านส่วนของตัวเองใน `docs/DEMO-SCRIPT.pdf` และเตรียมคำตอบจากตาราง "คำถามที่อาจโดนถาม"
2. ซ้อมทั้งชุดอย่างน้อย 1 รอบ — **ต้องก่อน 23:00** — จับเวลา (ประมาณ 12–15 นาที)

ช่วงที่ 5 — วัน Demo

👤 เรา (เครื่องที่ใช้นำเสนอ) — 5 นาทีก่อนเริ่ม

```
docker compose down -v
docker compose up -d --build
docker compose ps
chcp 65001
```

- Postman: Variables ต้องเป็น `localhost` (ถ้าเคยเปลี่ยนเป็น IP Radmin ให้เปลี่ยนกลับ)
- ขยายตัวอักษร PowerShell ด้วย **Ctrl + ลูกกลิ้งเมาส์**

ระหว่างนำเสนอ — เดินตาม `docs/DEMO-SCRIPT.pdf`

ลำดับ	ใคร	ส่วน
1	คนที่ 1	ภาพรวมระบบ + Auth
2	คนที่ 2	Service Management
3	คนที่ 3	Booking + คิววันนี้ ⭐
4	คนที่ 4	Notification & Report
5	คนที่ 1	ปิดท้าย

หลังจบ

```
docker compose stop
```

ถ้ายังมี rule firewall ค้างอยู่ ให้ลบด้วย `Remove-NetFirewallRule` ตามช่วงที่ 2

📌 ไฟล์อ้างอิง

ไฟล์	ใช้ทำอะไร
`docs/DEMO-SCRIPT.pdf`	สคริปต์พูดตอน demo + คำถามที่อาจโดนถาม
`docs/COMMANDS.pdf`	คำสั่งทั้งหมด + วิธีแก้ปัญหาที่เจอบ่อย
`README.md`	ภาพรวมระบบ และการตัดสินใจที่ spec ไม่ได้ระบุ
`postman/queue-booking.postman_collection.json`	ทดสอบ API และใช้ตอน demo